



Uniwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych

PROGRAMOWANIE OBIEKTOWE

Testownik - aplikacja do zarządzania testami i ocenami

Michał Nocek

Nr albumu: 134951

Kierunek studiów: Informatyka

Praca wykonana pod kierunkiem
mgr inż. Ewa Żesławska

Rzeszów, 2026

Spis treści

1. Streszczenie w języku polskim i angielskim	7
1.1. Streszczenie w języku polskim.....	7
1.2. Streszczenie w języku angielskim	7
2. Wprowadzenie i analiza problemu	8
2.1. Opis analizowanego problemu	8
2.2. Cel projektu	8
2.3. Zakres realizowanych funkcjonalności.....	8
2.4. Przegląd istniejących rozwiązań.....	9
2.4.1. Kahoot!	9
2.4.2. Microsoft Forms / Google Forms.....	9
2.4.3. Moduł testów w systemie Moodle	9
2.4.4. Wnioski z analizy.....	9
3. Wymagania systemowe	10
3.1. Wymagania funkcjonalne	10
3.2. Wymagania нефункционалне	11
3.3. Założenia dotyczące działania systemu.....	11
4. Architektura i struktura aplikacji.....	12
4.1. Architektura aplikacji	12
4.2. Struktura folderów i organizacja kodu	12
4.3. Zastosowane wzorce projektowe	12
4.4. Mechanizmy walidacji danych	13
4.5. Obsługa błędów i wyjątków	13
5. Technologie wykorzystane w projekcie	14
5.1. Technologie interfejsu użytkownika.....	14
5.2. Technologie dostępu do danych	14
5.3. Dodatkowe biblioteki i narzędzia	14
5.4. Uzasadnienie wyboru technologii	14
6. Projekt bazy danych	15
6.1. Opis modelu danych	15
6.2. Diagram ERD	15
6.3. Opis relacji i kluczy	16
6.4. Normalizacja.....	16
6.5. Migracje bazy danych.....	17
6.6. Mechanizmy zapewniające integralność danych.....	17
7. Implementacja aplikacji.....	18
7.1. Najważniejsze moduły systemu.....	18

7.2.	Mechanizm logowania i autoryzacji.....	20
7.3.	Realizacja operacji CRUD.....	20
7.4.	Logika oceniania	20
7.5.	Organizacja nawigacji	21
7.6.	Interfejs użytkownika i stylizacja	21
8.	Prezentacja warstwy użytkowej projektu.....	22
8.1.	Ekran logowania i rejestracji	22
8.2.	Pulpit nauczyciela.....	23
8.3.	Lista testów i edytor testu	23
8.4.	Ocenianie pytań otwartych	25
8.5.	Zarządzanie grupami	25
8.6.	Statystyki testu i eksport wyników	26
8.7.	Lista testów ucznia	27
8.8.	Rozwiązywanie testu	28
8.9.	Wynik testu	28
9.	Harmonogram realizacji projektu	30
9.1.	Napotkane trudności w trakcie realizacji projektu	30
9.2.	Repozytorium i system kontroli wersji.....	31
10.	Instrukcja uruchomienia projektu	32
10.1.	Wymagania systemowe	32
10.2.	Konfiguracja bazy danych	32
10.3.	Konfiguracja środowiska i uruchomienie aplikacji	32
10.4.	Dane logowania testowego	32
11.	Podsumowanie	33
11.1.	Ocena stopnia realizacji założeń	33
11.2.	Napotkane problemy	33
11.3.	Możliwe kierunki dalszego rozwoju.....	34
	Bibliografia	35
	Spis rysunków	36
	Spis tabel	37
	Spis listingów	38

1. Streszczenie w języku polskim i angielskim

1.1. Streszczenie w języku polskim

Praca opisuje aplikację *Testownik* - program do tworzenia i przeprowadzania testów oraz zarządzania wynikami uczniów. Celem projektu było danie nauczycielowi narzędzia, które obejmuje cały cykl pracy z testem: od ułożenia pytań, przez przypisanie testu grupom uczniów, po automatyczne i ręczne ocenianie odpowiedzi oraz analizę wyników.

Aplikacja została napisana w języku C# na platformie .NET z wykorzystaniem technologii *Windows Presentation Foundation* (WPF) do budowy interfejsu graficznego oraz *Entity Framework Core* jako warstwy dostępu do relacyjnej bazy danych PostgreSQL. System rozróżnia dwie role użytkowników - nauczyciela i ucznia - udostępniając każdej z nich odrębny zestaw funkcji. Nauczyciel tworzy testy złożone z pytań jednokrotnego wyboru, wielokrotnego wyboru oraz pytań otwartych, zarządza grupami uczniów i analizuje wyniki, natomiast uczeń rozwiązuje przypisane mu testy w ograniczonym czasie i przegląda swoje oceny. W aplikacji zaimplementowano między innymi limit podejść do testu, automatyczne ocenianie pytań zamkniętych, ręczne ocenianie i edycję oceny, sterowanie widocznością testów, wykrywanie prób oszustwa, eksport wyników do pliku CSV oraz haszowanie haseł. Efektem końcowym jest działająca aplikacja okienkowa gotowa do użycia.

1.2. Streszczenie w języku angielskim

This document describes the *Testownik* application - an information system for creating, sharing and conducting tests, as well as for managing students' results. The main goal of the project was to provide the teacher with a tool that automates the entire workflow of working with a test: from composing questions, through assigning the test to groups of students, up to automatic and manual grading of answers and the statistical analysis of results.

The application was written in C# on the .NET platform using *Windows Presentation Foundation* (WPF) for the graphical user interface and *Entity Framework Core* as the data access layer for a relational PostgreSQL database. The system distinguishes two user roles - the teacher and the student - providing each of them with a separate set of features. The teacher creates tests consisting of single-choice, multiple-choice and open questions, manages groups of students and analyses results, while the student solves the assigned tests within a limited time and reviews their grades. The application implements, among others, a limit on the number of attempts, automatic grading of closed questions, manual grading and grade editing, control over test visibility, cheating detection, exporting results to a CSV file and password hashing. The end result is a working desktop application ready for use.

2. Wprowadzenie i analiza problemu

2.1. Opis analizowanego problemu

Sprawdzanie wiedzy uczniów za pomocą testów jest jednym z podstawowych zadań nauczyciela, jednak w tradycyjnej, papierowej formie jest ono czasochłonne i mało wygodne. Wymaga ręcznego przygotowania arkuszy, ich powielenia, a następnie żmudnego sprawdzania odpowiedzi i podliczania punktów. Dodatkowo gromadzenie wyników oraz analiza tego, które zagadnienia sprawiają uczniom największe trudności, są w takim modelu praktycznie niemożliwe do przeprowadzenia na bieżąco.

Istnieją platformy do testów online, jednak najczęściej są to rozwiązania webowe, wymagające stałego dostępu do Internetu i kont w usługach zewnętrznych, a ich pełne możliwości są zwykle płatne. Brakuje prostego, samodzielnego narzędzia desktopowego, które łączyłoby tworzenie testów, ich automatyczne ocenianie oraz analizę wyników w jednym miejscu, z danymi przechowywanymi w lokalnej bazie.

2.2. Cel projektu

Celem projektu było stworzenie od podstaw funkcjonalnej aplikacji okienkowej w języku C#, współpracującej z relacyjną bazą danych, która realizuje pełny cykl pracy z testem:

- rejestrację i logowanie użytkowników z bezpiecznym przechowywaniem haseł oraz rozróżnieniem ról nauczyciela i ucznia,
- tworzenie testów złożonych z pytań jednokrotnego wyboru, wielokrotnego wyboru i pytań otwartych, z możliwością sterowania ich widocznością,
- przypisywanie testów grupom uczniów,
- rozwiązywanie testów przez uczniów w ograniczonym czasie, z obsługą wielu podejść,
- automatyczne ocenianie pytań zamkniętych oraz ręczne ocenianie i korektę punktów,
- wykrywanie prób oszustwa podczas rozwiązywania testu,
- analizę wyników w postaci statystyk oraz eksport do pliku CSV.

2.3. Zakres realizowanych funkcjonalności

Zakres projektu obejmuje następujące moduły:

1. **Moduł uwierzytelniania** - rejestracja (z wyborem roli), logowanie, haszowanie haseł, rozróżnienie ról nauczyciel/uczeń.
2. **Moduł zarządzania grupami** - tworzenie grup, zarządzanie ich składem, przynależność ucznia do wielu grup.
3. **Moduł testów** - tworzenie, edycja, duplikowanie i usuwanie testów, edytor pytań, ustawienia (czas, próg, liczba podejść), sterowanie widocznością (aktywny/nieaktywny).

4. **Moduł rozwiązywania testów** - rozwiązywanie w ograniczonym czasie, nawigacja między pytaniami, limit podejść, wykrywanie opuszczenia okna testu.
5. **Moduł oceniania** - automatyczne ocenianie pytań zamkniętych, ręczne ocenianie pytań otwartych oraz korekta oceny dowolnego podejścia.
6. **Moduł statystyk** - zdawalność, średni wynik, trudność pytań, tabela wyników, eksport do CSV.

2.4. Przegląd istniejących rozwiązań

Przed realizacją projektu przeanalizowano kilka istniejących narzędzi o podobnym przeznaczeniu.

2.4.1. Kahoot!

Popularna platforma do quizów i testów, działająca w przeglądarce, często wykorzystywana w szkołach. Jej zaletą jest grywalizacja i prostota, jednak jest to wyłącznie usługa internetowa, wymagająca stałego dostępu do sieci i konta, a obsługuje głównie pytania zamknięte. Pełne funkcje (m.in. rozbudowane raporty) są dostępne w wersji płatnej.

2.4.2. Microsoft Forms / Google Forms

Webowe narzędzia do tworzenia formularzy i quizów, umożliwiające automatyczne sprawdzanie odpowiedzi zamkniętych i zbieranie wyników. Są wygodne i darmowe w podstawowym zakresie, ale stanowią ogólne narzędzia do ankiet - brakuje im funkcji typowo dydaktycznych, takich jak limit podejść, grupy uczniów z przypisanymi testami, ręczne ocenianie pytań otwartych z korektą czy wykrywanie prób oszustwa. Wymagają też konta w usłudze chmurowej i połączenia z Internetem.

2.4.3. Moduł testów w systemie Moodle

Moodle to rozbudowana platforma e-learningowa oferująca zaawansowany moduł testów (różne typy pytań, banki pytań, statystyki). Jest jednak ciężka we wdrożeniu - wymaga serwera, bazy danych i konfiguracji całej platformy - co czyni ją nieproporcjonalnie złożoną do prostego scenariusza, w którym nauczyciel chce jedynie szybko ułożyć test i sprawdzić wyniki na własnym komputerze.

2.4.4. Wnioski z analizy

Żadne z analizowanych rozwiązań nie łączy lekkiej, samodzielnej aplikacji desktopowej z własną, lokalną bazą danych, automatycznym i ręcznym ocenianiem (w tym pytań otwartych), obsługą grup, limitem podejść, sterowaniem widocznością testu oraz wykrywaniem prób oszustwa - bez konieczności korzystania z usług chmurowych. Te cechy wyznaczyły zakres tego projektu.

3. Wymagania systemowe

3.1. Wymagania funkcjonalne

- **Zarządzanie kontami i rolami użytkowników**

- Użytkownik może samodzielnie zarejestrować nowe konto (imię, nazwisko, login, hasło), wybierając przy rejestracji rolę - ucznia lub nauczyciela. Login jest unikalny w skali całego systemu.
- System rozróżnia dwie role - nauczyciela i ucznia - i po zalogowaniu udostępnia każdej z nich odrębny zestaw funkcji oraz inny układ nawigacji. Każdy nauczyciel zarządza wyłącznie własnymi testami.

- **Zarządzanie grupami uczniów**

- Nauczyciel może tworzyć nowe grupy opatrzone nazwą i opisem oraz zarządzać ich składem.
- Jeden uczeń może należeć do wielu grup jednocześnie.

- **Tworzenie i edycja testów**

- Nauczyciel tworzy testy złożone z pytań trzech typów: jednokrotnego wyboru, wielokrotnego wyboru oraz pytań otwartych.
- Dla każdego testu określa nazwę, przedmiot, limit czasu, próg zaliczenia, dozwoloną liczbę podejść, widoczność oraz grupy, którym test jest udostępniany.
- Test można edytować, duplikować i usuwać. Po pojawieniu się rozwiązań edycja pytań jest blokowana, aby nie naruszyć powiązanych z nimi odpowiedzi.
- Tylko aktywny test jest widoczny dla uczniów; nowy test powstaje jako nieaktywny (szkic).

- **Rozwiązywanie testów**

- Uczeń widzi listę testów przypisanych do jego grup, z podziałem na rozwiązane i nierozwiązane.
- Podczas rozwiązywania działa licznik czasu; po jego upływie test jest automatycznie oddawany. Liczba podejść jest ograniczona wartością ustawioną przez nauczyciela.

- **Ocenianie odpowiedzi**

- Pytania zamknięte są oceniane automatycznie, w wielokrotnym wyborze z punktami częściowymi.
- Pytania otwarte ocenia ręcznie nauczyciel; może też skorygować punkty dowolnego pytania każdego podejścia. Przy wielu podejściach liczy się najlepszy wynik.

- **Wykrywanie prób oszustwa**

- Opuszczenie okna testu jest wykrywane: pierwsze skutkuje ostrzeżeniem, kolejne - zakończeniem testu z wynikiem 0% i oznaczeniem próby oszustwa.

- **Statystyki i eksport wyników**

- Nauczyciel ma dostęp do statystyk testu (zdawalność, średni wynik, najtrudniejsze pytanie, poprawność pytań) oraz może wyeksportować wyniki do pliku CSV.

3.2. Wymagania niefunkcjonalne

Bezpieczeństwo

- Dostęp do funkcji systemu jest możliwy wyłącznie po zalogowaniu, a zakres uprawnień zależy od roli. Hasła nie są przechowywane jawnie - zapisuje się ich skrót (SHA-256). Unikalność loginów jest wymuszana na poziomie bazy danych.

Niezawodność i obsługa błędów

- Operacje na bazie danych oraz operacje wejścia/wyjścia są zabezpieczone wspólnym mechanizmem obsługi wyjątków. W razie błędu użytkownik otrzymuje zrozumiały komunikat, a aplikacja nie ulega awarii.

Spójność danych

- Integralność jest wymuszana kluczami obcymi z odpowiednimi regułami usuwania, unikalnym indeksem oraz mechanizmem migracji wersjonującym schemat bazy.

Użyteczność

- Interfejs jest czytelny i spójny, nawigacja odbywa się przez stały panel boczny, a komunikaty prezentowane są w języku polskim. Nowy użytkownik bez trudu odnajduje podstawowe funkcje.

Wydajność

- Kluczowe operacje (logowanie, wczytywanie testów, zapis wyniku) wykonują się w czasie nieprzekraczającym kilku sekund.

3.3. Założenia dotyczące działania systemu

Przyjęto następujące założenia:

- Aplikacja działa w systemie Windows 10/11 z zainstalowanym środowiskiem uruchomieniowym .NET.
- Dostępna jest instancja serwera PostgreSQL; baza danych i jej schemat są tworzone automatycznie przy pierwszym uruchomieniu za pomocą migracji Entity Framework Core.
- Dane początkowe (przykładowy nauczyciel, uczniowie, grupy i testy) są wstawiane automatycznie, co umożliwia natychmiastowe przetestowanie aplikacji.
- Każdy nauczyciel pracuje na własnym zbiorze testów; grupy uczniów są zasobem współdzielonym.

4. Architektura i struktura aplikacji

4.1. Architektura aplikacji

Aplikacja ma architekturę warstwową. Wyróżniono cztery warstwy o odrębnych odpowiedzialnościach:

1. **Warstwa modelu (Models)** - klasy encji odwzorowujące obiekty dziedziny (użytkownik, grupa, test, pytanie, podejście, odpowiedź), mapowane na tabele bazy danych przez Entity Framework Core.
2. **Warstwa dostępu do danych (Data)** - kontekst bazy danych (AppDbContext) z konfiguracją relacji i dziedziczenia, mechanizm migracji oraz klasa zasilająca bazę danymi początkowymi (DbSeeder).
3. **Warstwa logiki i usług (Services)** - repozytorium operacji bazodanowych (DbRepository), serwis oceniania (GradingService), eksport do CSV (CsvExporter), haszowanie haseł (PasswordHash) oraz wspólna obsługa wyjątków (Safe). Tu znajdują się także obiekty transferu danych (DTO).
4. **Warstwa prezentacji (Views)** - okna i ekrany aplikacji zbudowane w technologii WPF, odpowiedzialne za wyświetlanie danych i obsługę interakcji użytkownika.

Przepływ jest jednokierunkowy: widok wywołuje warstwę usług, ta korzysta z repozytorium, a repozytorium komunikuje się z bazą przez kontekst Entity Framework Core. Widoki nigdy nie odwołują się do bazy bezpośrednio - zawsze pośredniczy w tym warstwa usług.

4.2. Struktura folderów i organizacja kodu

Kod źródłowy podzielono na foldery odpowiadające warstwom aplikacji (listing 4.1).

Listing 4.1. Struktura katalogów projektu.

```
1 ProjektPO2/
2 |-- App.xaml(.cs)           // punkt startowy aplikacji
3 |-- MainWindow.xaml(.cs)    // powłoka: panel boczny + obszar treści
4 |-- Theme.xaml             // motyw: kolory, style kontroltek
5 |-- Models/                // encje (User, Quiz, Question, ...)
6 |-- Data/                  // AppDbContext, DbSeeder, fabryka kontekstu
7 |-- Migrations/            // migracje EF Core
8 |-- Services/              // DbRepository, GradingService, CsvExporter,
9 |                           // PasswordHash, Safe, DTO (AppModels)
10 |-- Views/                 // ekrany aplikacji (Login, Teacher*, Quiz, ...)
11 \-- Commands/             // RelayCommand
```

4.3. Zastosowane wzorce projektowe

W projekcie wykorzystano kilka rozwiązań i wzorców porządkujących kod:

- **Architektura warstwowa** - oddzielenie modelu, dostępu do danych, logiki i prezentacji.
- **Repozytorium (Repository)** - klasa DbRepository hermetyzuje cały dostęp do bazy danych i odcina od niej resztę aplikacji.

- **Obiekty transferu danych (DTO)** - lekkie klasy w warstwie usług oddzielają dane prezentowane w widokach od encji bazodanowych.
- **Dziedziczenie i polimorfizm** - trzy hierarchie klas abstrakcyjnych (użytkownicy, pytania, odpowiedzi) mapowane na bazę strategią Table-per-Hierarchy.
- **Interfejs IPublishable** - implementowany przez klasę `Quiz`, ujednolica sterowanie widocznością testu (metody `Publish()` i `Unpublish()`).

4.4. Mechanizmy walidacji danych

Walidacja realizowana jest na trzech poziomach:

1. **Interfejs użytkownika** - pola formularzy są sprawdzane przed zapisem: niepuste wymagane pola, liczby ograniczane do dozwolonego zakresu (np. czas, próg, liczba podejść), a edytor testu blokuje zapis testu niekompletnego (pytanie bez poprawnej odpowiedzi, puste pola, brak przypisanej grupy).
2. **Logika aplikacji** - serwis oceniania pilnuje zakresu przyznawanych punktów, a repozytorium dba o spójność operacji (np. bezpieczną kolejność usuwania powiązanych danych).
3. **Baza danych** - ograniczenia na poziomie tabel (klucze obce, unikalny indeks) zapewniają spójność nawet przy bezpośrednim dostępie do bazy.

4.5. Obsługa błędów i wyjątków

Operacje na bazie danych oraz operacje wejścia/wyjścia są wykonywane za pośrednictwem wspólnej klasy `Safe`, która przechwytuje wyjątki i prezentuje użytkownikowi czytelny komunikat zamiast powodować awarię aplikacji. Dzięki temu np. brak połączenia z bazą czy błąd zapisu pliku CSV nie zatrzymuje programu, a użytkownik dowiaduje się o przyczynie problemu.

5. Technologie wykorzystane w projekcie

5.1. Technologie interfejsu użytkownika

Warstwę prezentacji zrealizowano w technologii *Windows Presentation Foundation* (WPF) [3] na platformie .NET, w języku C# [1]. WPF posłużył do budowy okien aplikacji, paska bocznego nawigacji oraz ekranów poszczególnych modułów. Wygląd aplikacji (kolory, style kontrolek) zdefiniowano w słowniku zasobów `Theme.xaml`, co oddziela warstwę wizualną od logiki. Strona logowania zbudowana jest deklaratywnie w pliku XAML, natomiast pozostałe ekrany tworzone są dynamicznie w kodzie, z wykorzystaniem wspólnych metod pomocniczych budujących powtarzalne elementy interfejsu (karty, znaczniki statusu, wykresy).

5.2. Technologie dostępu do danych

Warstwę dostępu do danych oparto na bibliotece *Entity Framework Core* [2] - mechanizmie mapowania obiektowo-relacyjnego (ORM). Pozwala on operować na bazie za pomocą obiektów języka C# i zapytań LINQ, bez ręcznego pisania większości zapytań SQL. Systemem zarządzania bazą danych jest *PostgreSQL* [5], łączony z aplikacją za pośrednictwem dostawcy *Npgsql* [4]. Schemat bazy jest wersjonowany i tworzony automatycznie poprzez migracje *Entity Framework Core*.

5.3. Dodatkowe biblioteki i narzędzia

W projekcie wykorzystano ponadto:

- `System.Security.Cryptography` - moduł platformy .NET użyty do wyznaczania skrótu SHA-256 haseł użytkowników.
- `System.IO (StreamWriter)` - strumieniowy zapis pliku CSV z wynikami testu, z kodowaniem UTF-8 obsługującym polskie znaki.
- *Visual Studio* - środowisko programistyczne wykorzystane do budowy i uruchamiania aplikacji.
- *pgAdmin* - narzędzie do zarządzania bazą danych PostgreSQL.
- *Git* - system kontroli wersji projektu.

5.4. Uzasadnienie wyboru technologii

WPF, *Entity Framework Core* i *PostgreSQL* dobrano pod charakter aplikacji:

- **WPF** wybrano, bo aplikacja jest programem desktopowym z rozbudowanym interfejsem (karty, wykresy, stylizacja), a nie potrzebuje serwera ani przeglądarki.
- **Entity Framework Core** wybrano zamiast bezpośredniego ADO.NET, ponieważ model danych opiera się na dziedziczeniu (trzy hierarchie klas) i wielu relacjach. ORM odwzorowuje je automatycznie (strategią Table-per-Hierarchy), a migracje pozwalają wersjonować schemat bez ręcznego pisania skryptów SQL, co skraca kod warstwy danych.
- **PostgreSQL** to darmowy, otwarty i niezawodny system bazodanowy, dobrze wspierany przez *Entity Framework Core* poprzez dostawcę *Npgsql*.

6. Projekt bazy danych

6.1. Opis modelu danych

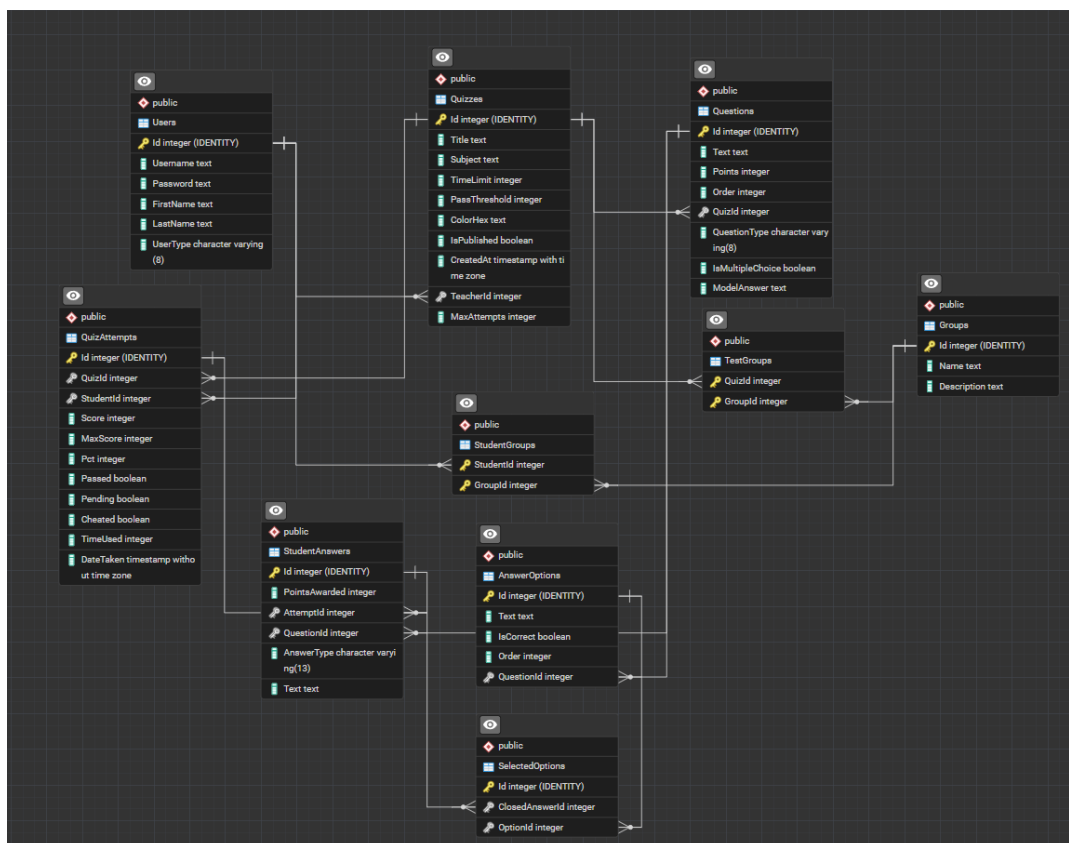
Wszystkie informacje o użytkownikach, grupach, testach, pytaniach, podejściach i odpowiedziach są przechowywane w relacyjnej bazie danych PostgreSQL. Najważniejsze tabele przedstawiono w tabeli 6.1. Można je podzielić na trzy obszary: konta użytkowników i grupy, testy wraz z pytaniami oraz podejścia uczniów wraz z ich odpowiedziami.

Tabela 6.1. Najważniejsze tabele bazy danych aplikacji.

Tabela	Przeznaczenie
Users	Konta użytkowników (wspólna tabela nauczycieli i uczniów, rozróżnianych dyskriminatorem); login, skrót hasła, imię i nazwisko.
Groups	Grupy uczniów (nazwa, opis).
StudentGroups	Tabela łącząca uczniów z grupami (relacja wiele-do-wielu).
Quizzes	Testy (nazwa, przedmiot, limit czasu, próg, liczba podejść, widoczność).
Questions	Pytania (wspólna tabela pytań zamkniętych i otwartych); treść, punkty, kolejność.
AnswerOptions	Warianty odpowiedzi pytań zamkniętych; treść, flaga poprawności.
TestGroups	Tabela łącząca testy z grupami (relacja wiele-do-wielu).
QuizAttempts	Podejścia uczniów do testów; wynik, procent, status, czas, data.
StudentAnswers	Odpowiedzi ucznia (wspólna tabela odpowiedzi zamkniętych i otwartych).
SelectedOptions	Warianty zaznaczone przez ucznia w odpowiedzi zamkniętej.

6.2. Diagram ERD

Centralnymi tabelami modelu są `Quizzes` (testy) oraz `Users` (użytkownicy). Test posiada pytania i podejścia, a podejście ucznia gromadzi jego odpowiedzi. Pełny schemat powiązań przedstawia diagram ERD na rysunku 6.1.



Rysunek 6.1. Diagram ERD bazy danych aplikacji.

6.3. Opis relacji i kluczy

Każda tabela posiada klucz główny typu całkowitego (`IDENTITY`). Główne relacje:

- `Quizzes` - `Questions` oraz `Quizzes` - `QuizAttempts`: relacje jeden-do-wiele (test ma wiele pytań i wiele podejść).
- `ClosedQuestion` - `AnswerOptions`: pytanie zamknięte ma wiele wariantów odpowiedzi.
- `QuizAttempts` - `StudentAnswers` - `SelectedOptions`: podejście ma wiele odpowiedzi, a odpowiedź zamknięta - wiele zaznaczonych wariantów.
- `StudentGroups` łączy uczniów i grupy w relacji wiele-do-wiele (klucz złożony `StudentId` + `GroupId`); analogicznie `TestGroups` łączy testy z grupami.

6.4. Normalizacja

Baza danych spełnia trzecią postać normalną (3NF) - każda tabela przechowuje dane dotyczące jednego pojęcia, wartości w kolumnach są atomowe, a powtarzające się informacje zostały zastąpione kluczami obcymi. Relacje wiele-do-wiele (uczeń-grupa oraz test-grupa) rozwiązano za pomocą tabel łączących (`StudentGroups`, `TestGroups`), co eliminuje powielanie danych. Dziedziczenie klas odwzorowano strategią Table-per-Hierarchy, w której cała hierarchia mieści się w jednej tabeli, a typ rekordu wskazuje kolumna dyskryminatora.

6.5. Migracje bazy danych

Zmiany schematu bazy są zapisane w postaci migracji Entity Framework Core w katalogu Migrations. Przy starcie aplikacja wywołuje metodę `Database.Migrate()`, która tworzy bazę i nakłada wszystkie migracje. Kolejne migracje wprowadzały m.in. obsługę liczby podejść, zmianę typu daty oddania na pełny znacznik czasu, unikalny indeks loginu oraz haszowanie haseł istniejących kont. Dzięki migracjom schemat jest wersjonowany, a baza odtwarzalna bez ręcznego pisania skryptów SQL.

6.6. Mechanizmy zapewniające integralność danych

- **Integralność referencyjna** - klucze obce na wszystkich powiązaniach. Tam, gdzie dane podrzędne nie mają sensu bez nadrzędnych, zastosowano usuwanie kaskadowe (usunięcie testu usuwa jego pytania i podejścia); tam, gdzie usunięcie mogłoby naruszyć już oddane odpowiedzi, zastosowano regułę ograniczającą, a operacje usuwania wykonywane są we właściwej kolejności (najpierw rekordy zależne).
- **Unikalność** - unikalny indeks na loginie użytkownika gwarantuje brak dwóch kont o tej samej nazwie.
- **Spójność dziedzinowa** - walidacja po stronie aplikacji nie dopuszcza do zapisu testu niekompletnego (np. pytania bez poprawnej odpowiedzi).

7. Implementacja aplikacji

7.1. Najważniejsze moduły systemu

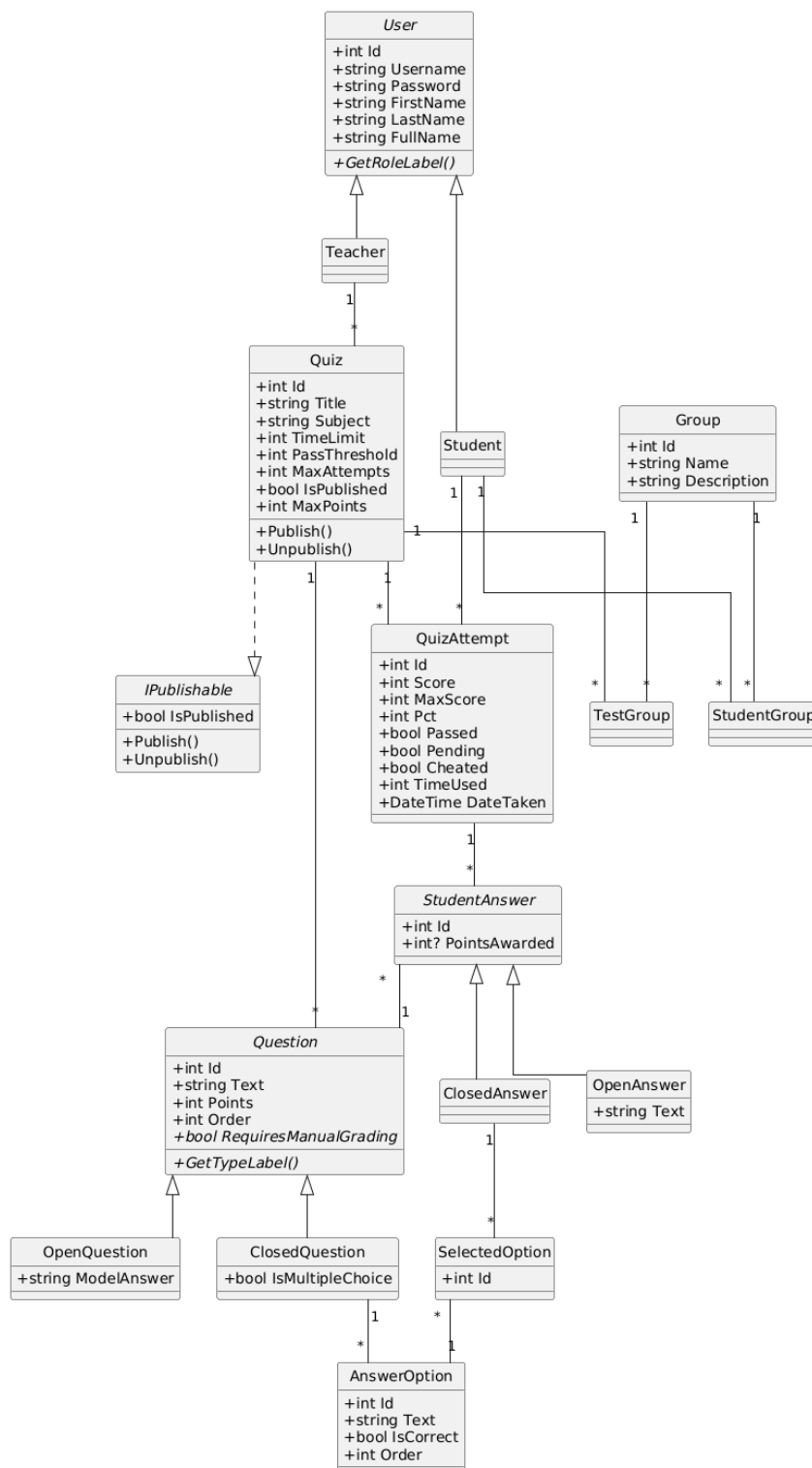
Logika aplikacji skupiona jest w warstwie usług. Kluczowe klasy to:

- `DbRepository` - centralne repozytorium operacji na bazie danych; pobieranie i zapisywanie testów, grup, podejść i wyników oraz mapowanie encji na obiekty DTO,
- `GradingService` - logika oceniania (punkty za pojedyncze pytanie oraz przeliczanie całego wyniku podejścia),
- `CsvExporter` - eksport wyników testu do pliku CSV,
- `PasswordHash` - haszowanie haseł,
- `Safe` - wspólna obsługa wyjątków.

W modelu danych szeroko wykorzystano dziedziczenie - występują trzy hierarchie klas abstrakcyjnych:

- abstrakcyjna klasa `User` przechowuje wspólne dane konta (login, skrót hasła, imię i nazwisko) oraz metodę abstrakcyjną `GetRoleLabel`; dziedziczą po niej `Teacher` oraz `Student`,
- abstrakcyjna klasa `Question` (treść, punkty, kolejność, właściwość `RequiresManualGrading`) jest bazą dla `ClosedQuestion` - pytania zamkniętego z wariantami odpowiedzi - oraz `OpenQuestion` - pytania otwartego z odpowiedzią wzorcową,
- abstrakcyjna klasa `StudentAnswer` jest bazą dla `ClosedAnswer` (przechowuje zaznaczone warianty) i `OpenAnswer` (przechowuje wpisany tekst).

Dziedziczenie pozwala m.in. trzymać w jednej kolekcji pytania lub odpowiedzi obu typów i obsługiwać je polimorficznie. Dodatkowo klasa `Quiz` implementuje interfejs `IPublishable`, ujednolicający sterowanie widocznością testu. Wszystkie trzy hierarchie są mapowane na bazę danych strategią Table-per-Hierarchy. Wzajemne powiązania najważniejszych klas modelu przedstawia diagram UML na rysunku 7.1.



Rysunek 7.1. Diagram UML klas modelu aplikacji.

7.2. Mechanizm logowania i autoryzacji

Hasła nie są przechowywane jawnie - zapisywany jest ich skrót SHA-256 w postaci szesnastkowej. Podczas logowania wyznaczany jest skrót wpisanego hasła i porównywany ze skrótem w bazie (listing 7.1). Autoryzacja oparta jest na roli użytkownika: po zalogowaniu obiekt użytkownika trafia do okna głównego, które buduje inny zestaw modułów dla nauczyciela, a inny dla ucznia.

Listing 7.1. Haszowanie hasła i logowanie.

```
1 public static string Hash(string password)
2 {
3     byte[] bytes = SHA256.HashData(Encoding.UTF8.GetBytes(password));
4     return Convert.ToString(bytes).ToLowerInvariant();
5 }
6
7 // logowanie - porównanie skrotu wpisanego hasla ze skrotem w bazie
8 var hash = PasswordHash.Hash(password);
9 var user = ctx.Users.FirstOrDefault(
10     x => x.Username == username && x.Password == hash);
```

7.3. Realizacja operacji CRUD

Operacje tworzenia, odczytu, modyfikacji i usuwania (CRUD) realizowane są przez repozytorium `DbRepository` z użyciem `Entity Framework Core`. Przykładem nietrywialnej operacji jest usuwanie testu (listing 7.2). Ponieważ odpowiedzi uczniów są powiązane z pytaniami kluczem obcym w trybie ograniczającym (`Restrict`), bezpośrednie usunięcie testu mogłoby naruszyć integralność. Dlatego najpierw usuwane są odpowiedzi i podejścia, a dopiero potem sam test - pozostałe dane (pytania, opcje, przypisania grup) znikają kaskadowo.

Listing 7.2. Bezpieczne usuwanie testu (zachowanie integralności).

```
1 public static void DeleteTest(int id)
2 {
3     using var ctx = Ctx();
4     var quiz = ctx.Quizzes
5         .Include(q => q.Attempts).ThenInclude(a => a.Answers)
6         .FirstOrDefault(q => q.Id == id);
7     if (quiz == null) return;
8
9     foreach (var a in quiz.Attempts)
10         ctx.StudentAnswers.RemoveRange(a.Answers);
11     ctx.QuizAttempts.RemoveRange(quiz.Attempts);
12     ctx.Quizzes.Remove(quiz);
13     ctx.SaveChanges();
14 }
```

7.4. Logika oceniania

Najważniejszym elementem logiki jest serwis oceniania. Metoda `Grade` wyznacza liczbę punktów za pojedyncze pytanie (listing 7.3). Pierwszeństwo ma ręcznie przyznana ocena - dotyczy to każdego typu pytania, dzięki czemu nauczyciel może nadpisać również automatyczną ocenę pytania zamkniętego. W przypadku braku ręcznej oceny pytania zamknięte są liczone automatycznie, przy czym w pytaniach wielokrotnego wyboru przyznawane są punkty częściowe (proporcjonalnie do liczby trafień pomniejszych)

o błędne wskazania). Pytania otwarte bez ręcznej oceny oczekują na nauczyciela. Cały wynik podejścia przeliczany jest następnie metodą `Recompute`, a przy wielu podejściach do oceny brany jest najlepszy wynik ucznia.

Listing 7.3. Ocenianie pojedynczego pytania (`GradingService`).

```
1 public static (int pts, bool ok, bool partial, bool pending) Grade(  
2     Question q, object? ans, IReadOnlyDictionary<int, int>? manual = null)  
3 {  
4     // ręczne nadpisanie punktów ma pierwszeństwo (każdy typ pytania)  
5     if (manual != null && manual.TryGetValue(q.Id, out var mo))  
6         return (mo, mo >= q.Points, mo > 0 && mo < q.Points, false);  
7  
8     switch (q.Type)  
9     {  
10        case QuestionType.Multi:  
11            var chosen = AsIndices(ans);  
12            int good = chosen.Count(i => q.Correct.Contains(i));  
13            int bad = chosen.Count(i => !q.Correct.Contains(i));  
14            double raw = q.Correct.Count == 0 ? 0  
15                : Math.Max(0, good - bad) / (double)q.Correct.Count;  
16            int pts = (int)Math.Round(raw * q.Points);  
17            return (pts, pts == q.Points && q.Points > 0,  
18                pts > 0 && pts < q.Points, false);  
19            // ... pytania jednokrotnego wyboru i otwarte  
20        }  
21        return (0, false, false, false);  
22    }
```

7.5. Organizacja nawigacji

Główne okno (`MainWindow`) pełni rolę powłoki: zawiera stały panel boczny z nawigacją oraz obszar treści, w którym wyświetlane są kolejne ekrany modułów. Po kliknięciu pozycji menu tworzona jest odpowiednia strona i wstawiana do obszaru treści. Ekran rozwiązywania testu oraz ekran wyniku wyświetlane są w trybie pełnoekranowym (bez panelu bocznego), aby skupić uwagę ucznia na zadaniu.

7.6. Interfejs użytkownika i stylizacja

Wygląd aplikacji zdefiniowano w słowniku zasobów `Theme.xaml`, obejmującym paletę kolorów oraz style przycisków, pól i innych kontroltek. Powtarzalne elementy interfejsu (karty, znaczniki statusu, wykresy kołowe i słupkowe, awatary) budowane są przez wspólne metody pomocnicze, dzięki czemu ekrany wyglądają jednolicie. Okno główne otwiera się zmaksymalizowane, a układ oparty na siatkach dopasowuje się do rozmiaru okna.

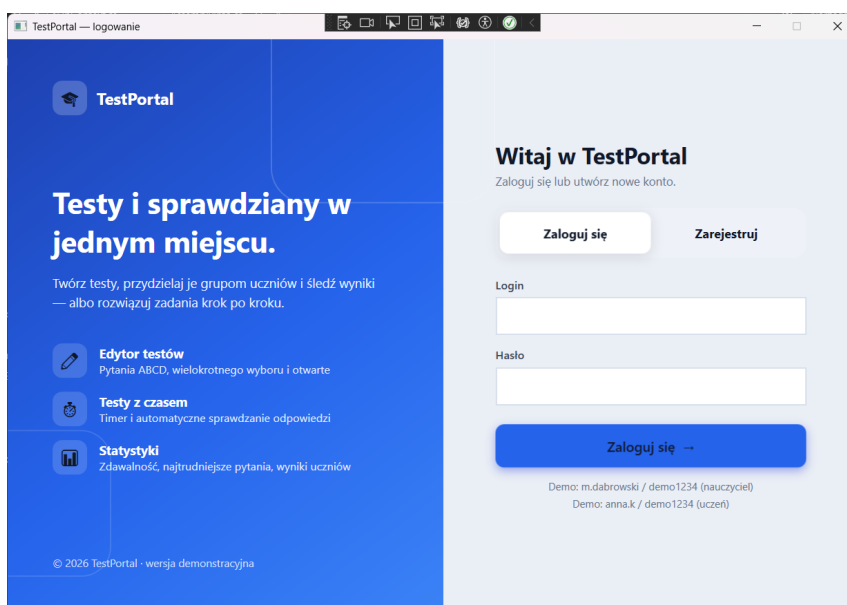
8. Prezentacja warstwy użytkowej projektu

W tym rozdziale przedstawiono wizualną stronę aplikacji za pomocą zrzutów ekranu, ilustrujących kluczowe widoki oraz interakcje użytkownika. Celem jest zaprezentowanie funkcjonalności systemu z perspektywy użytkownika końcowego - zarówno nauczyciela, jak i ucznia. Kolejne podrozdziały odpowiadają typowemu scenariuszowi pracy: nauczyciel loguje się, tworzy i udostępnia test, uczeń rozwiązuje przypisany mu test, a następnie nauczyciel ocenia odpowiedzi i analizuje wyniki.

8.1. Ekran logowania i rejestracji

Pierwszym ekranem, z którym styka się użytkownik, jest ekran logowania, umożliwiający dostęp do systemu zarejestrowanym użytkownikom lub utworzenie nowego konta (ucznia lub nauczyciela). Interfejs przedstawiono na rysunku 8.1. Składa się on z:

- pola *Login*, służącego do wprowadzenia nazwy użytkownika,
- pola *Hasło*, zapewniającego maskowanie wpisywanego hasła,
- przycisku *Zaloguj się*, który weryfikuje podane dane z bazą danych i w przypadku ich poprawności przenosi użytkownika do odpowiedniego panelu (nauczyciela lub ucznia),
- zakładki *Rejestracja*, umożliwiającej utworzenie nowego konta po podaniu imienia, nazwiska, loginu i hasła oraz wybraniu roli (ucznia lub nauczyciela),
- komunikatów zwrotnych, informujących o błędnych danych logowania lub o problemach z walidacją formularza rejestracji.

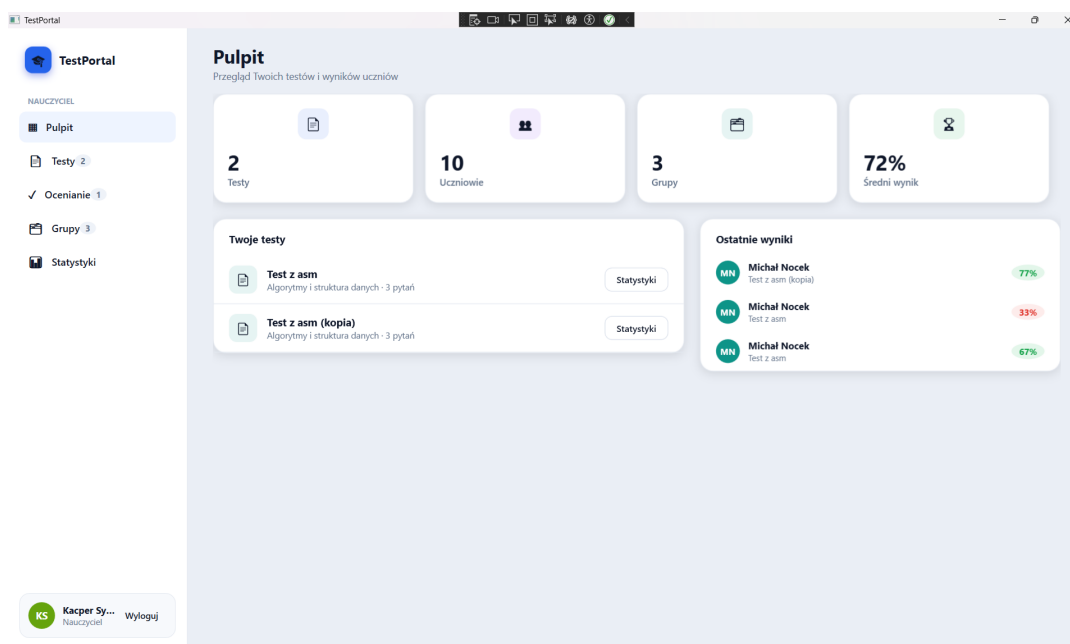


Rysunek 8.1. Ekran logowania i rejestracji.

8.2. Pulpit nauczyciela

Po zalogowaniu na konto nauczyciela wyświetlany jest pulpit (rysunek 8.2), stanowiący przegląd najważniejszych informacji. Zawiera on:

- kafelki podsumowujące liczbę testów, uczniów i grup oraz średni wynik,
- listę testów nauczyciela z szybkim przejściem do ich statystyk,
- panel ostatnich wyników uczniów,
- boczny panel nawigacji umożliwiający przejście do pozostałych modułów: testów, oceniania, grup i statystyk.

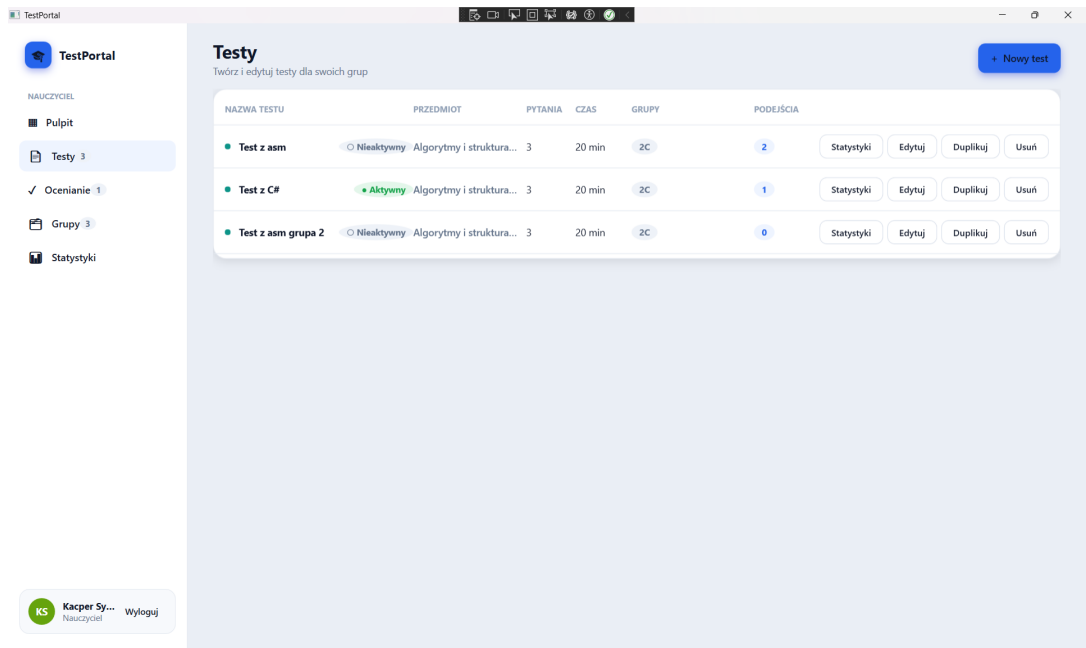


Rysunek 8.2. Pulpit nauczyciela.

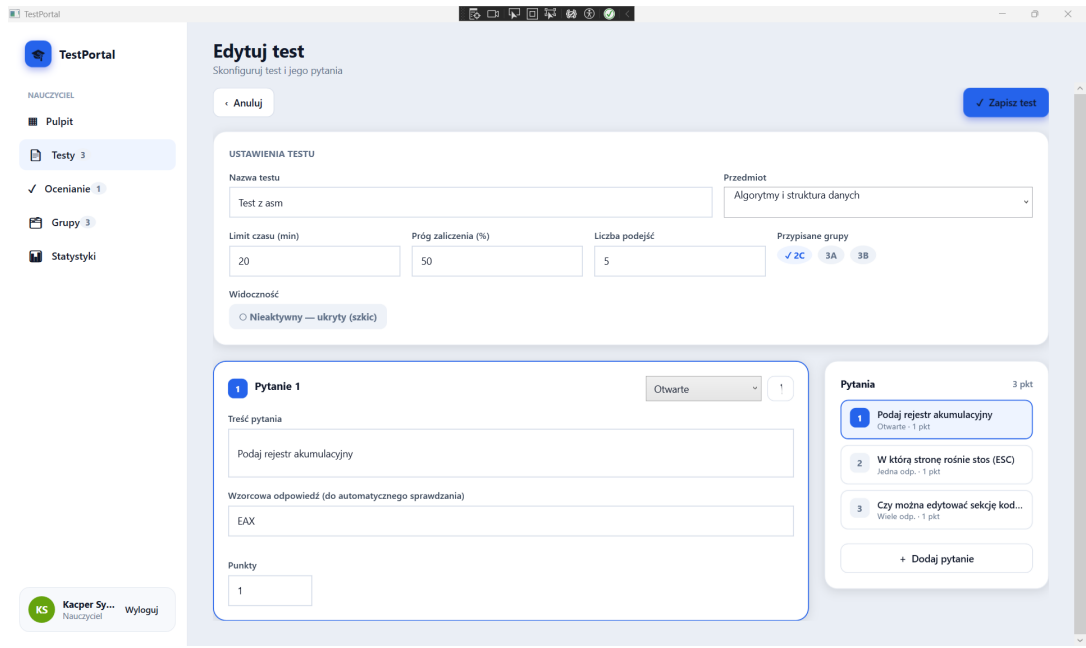
8.3. Lista testów i edytor testu

Widok *Testy* prezentuje wszystkie testy nauczyciela w formie tabeli z informacjami o nazwie i statusie widoczności (aktywny/nieaktywny), przedmiocie, liczbie pytań, czasie, przypisanych grupach i liczbie podejść. Status można przełączać jednym kliknięciem, a przy każdym teście dostępne są akcje: statystyki, edycja, duplikowanie oraz usunięcie. Z poziomu tego widoku nauczyciel tworzy nowy test lub otwiera istniejący w edytorze (rysunek 8.4).

Edytor testu pozwala określić ustawienia testu (nazwę, przedmiot wpisywany dowolnie lub wybierany z listy, limit czasu, próg zaliczenia, liczbę podejść, widoczność dla uczniów oraz przypisane grupy) i zbudować listę pytań. Dla każdego pytania można wybrać typ (jednokrotny wybór, wielokrotny wybór lub pytanie otwarte), wpisać treść, ustalić liczbę punktów oraz - w pytaniach zamkniętych - warianty odpowiedzi i poprawne odpowiedzi. Edytor waliduje kompletność testu przed zapisem (m.in. obecność poprawnej odpowiedzi i przypisanej grupy). Jeżeli test ma już rozwiązania, edycja pytań jest blokowana, o czym informuje stosowny komunikat.



Rysunek 8.3. Lista testów nauczyciela ze statusem widoczności i akcjami.



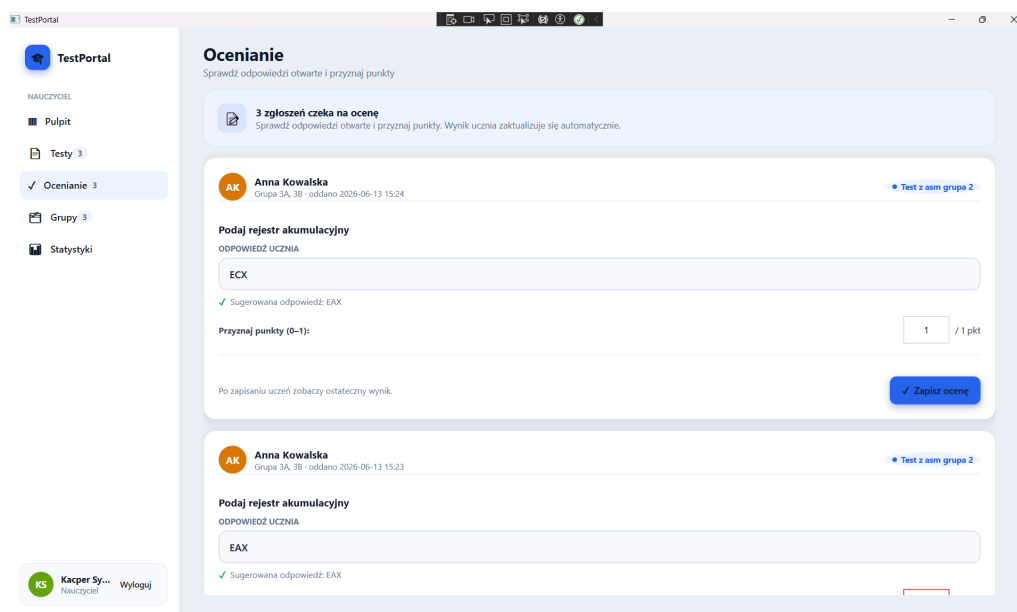
Rysunek 8.4. Edytor testu wraz z ustawieniami i listą pytań.

8.4. Ocenianie pytań otwartych

Moduł *Ocenianie* (rysunek 8.5) gromadzi podejścia uczniów zawierające pytania otwarte oczekujące na ręczną ocenę. Dla każdego zgłoszenia nauczyciel widzi:

- dane ucznia oraz nazwę testu i datę oddania,
- treść pytania otwartego oraz odpowiedź wpisaną przez ucznia,
- podpowiedź w postaci odpowiedzi wzorcowej,
- pole do wpisania przyznanych punktów w dozwolonym zakresie (z walidacją zakresu).

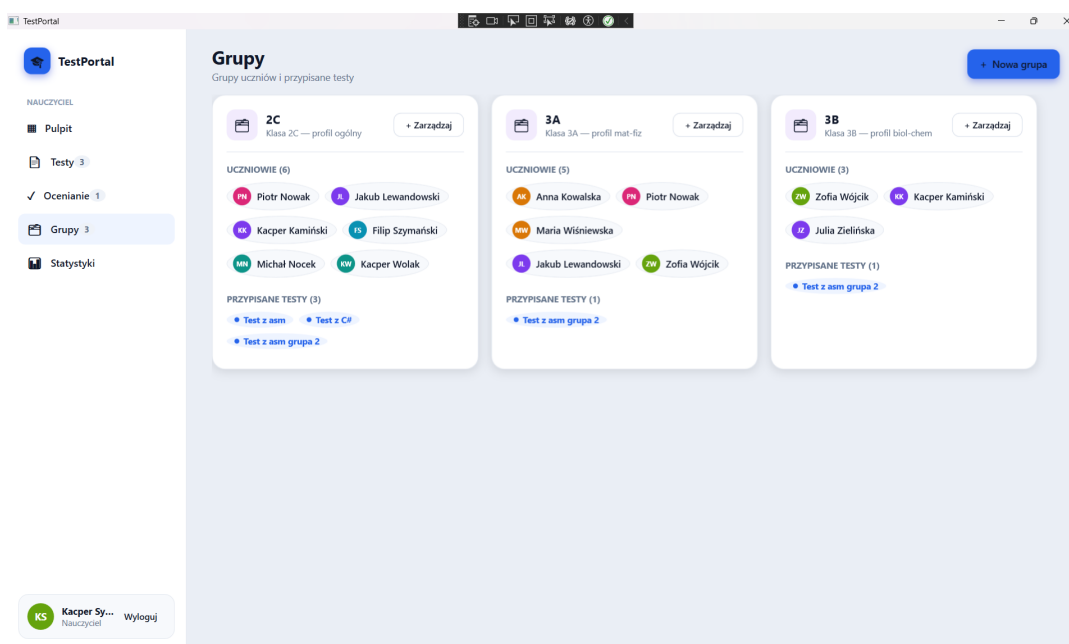
Po przyznaniu punktów wszystkim pytaniom otwartym i zapisaniu oceny wynik ucznia jest automatycznie przeliczany, a zgłoszenie znika z kolejki.



Rysunek 8.5. Ocenianie pytań otwartych.

8.5. Zarządzanie grupami

Widok *Grupy* (rysunek 8.6) prezentuje grupy uczniów w formie kart. Każda karta zawiera nazwę i opis grupy, listę przypisanych uczniów oraz listę testów udostępnionych grupie. Przycisk zarządzania otwiera okno, w którym nauczyciel zaznacza uczniów należących do grupy. Dodatkowo, z poziomu tego widoku można utworzyć nową grupę.



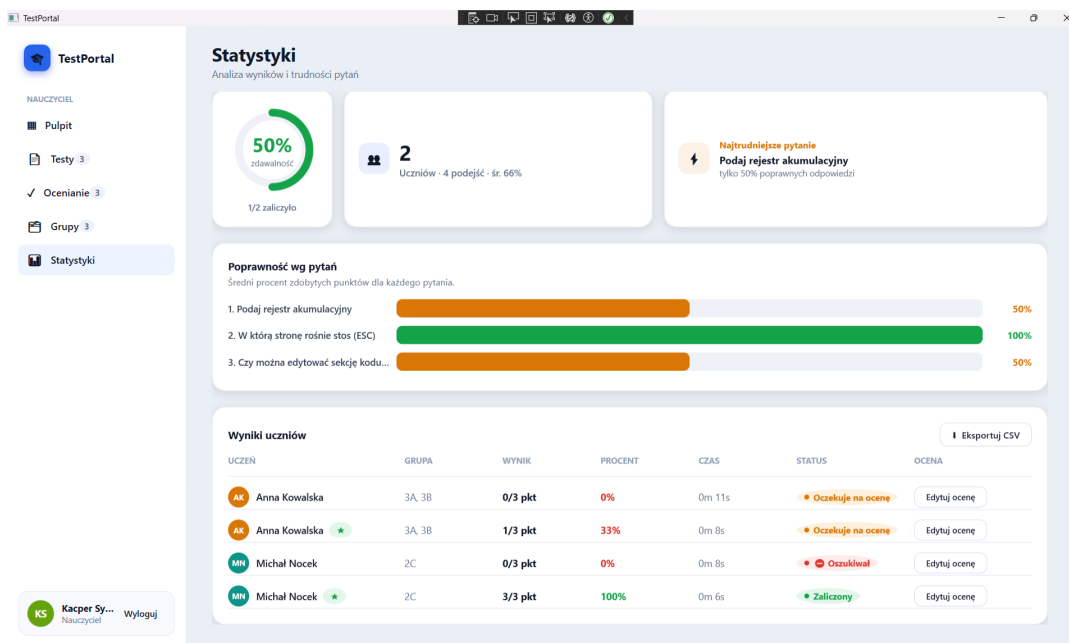
Rysunek 8.6. Widok zarządzania grupami uczniów.

8.6. Statystyki testu i eksport wyników

Moduł *Statystyki* (rysunek 8.7) prezentuje analizę wyników wybranego testu. Zawiera:

- wskaźnik zdawalności oraz liczbę uczniów, łączną liczbę podejść i średni wynik,
- informację o najtrudniejszym pytaniu,
- wykres poprawności w rozbiciu na poszczególne pytania,
- tabelę z wszystkimi podejściami uczniów (najlepsze podejście oznaczone gwiazdką) wraz ze statusem (zaliczony, niezaliczony, oczekujący na ocenę, próba oszustwa) oraz możliwością ręcznej edycji oceny każdego podejścia,
- przycisk eksportu wyników do pliku CSV.

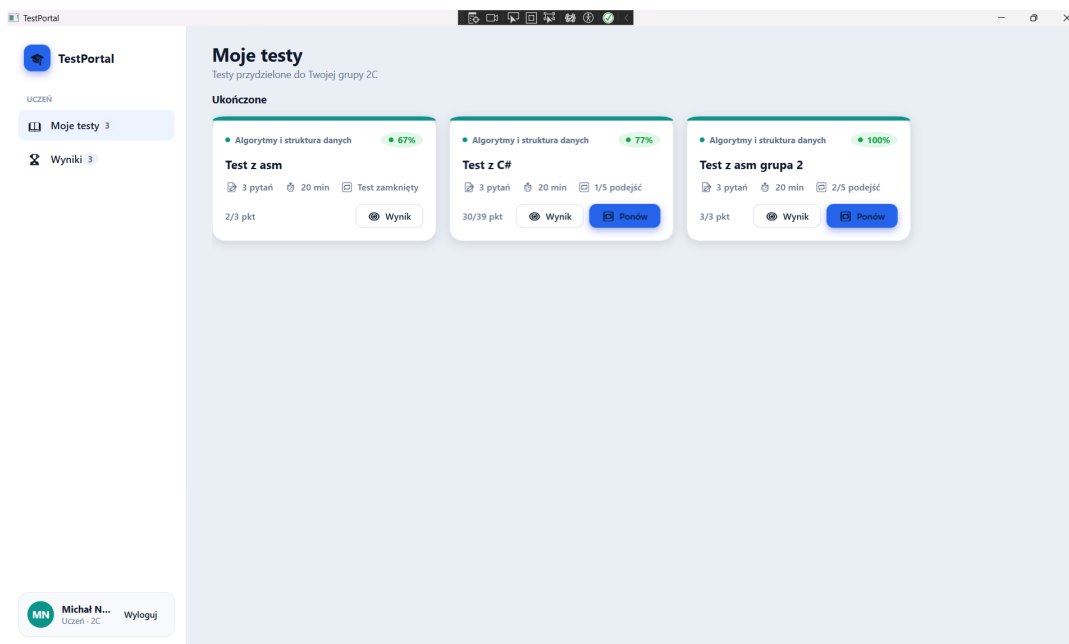
Wskaźniki zbiorcze (zdawalność, średni wynik) liczone są z najlepszego podejścia każdego ucznia.



Rysunek 8.7. Statystyki wyników testu.

8.7. Lista testów ucznia

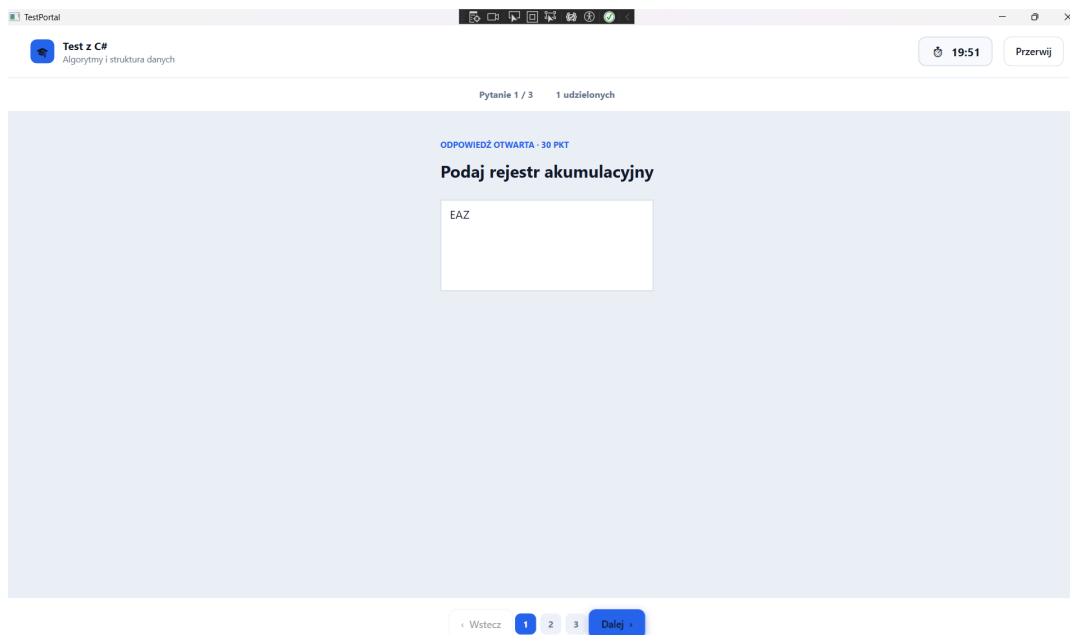
Po zalogowaniu na konto ucznia wyświetlana jest lista testów przypisanych do jego grup (rysunek 8.8), z podziałem na testy do zrobienia oraz ukończone. Każdy test prezentowany jest w formie karty z informacją o przedmiocie, liczbie pytań, czasie, liczbie wykorzystanych podejść oraz - dla testów rozwiązanych - z osiągniętym wynikiem i możliwością jego podglądu lub ponownego podejścia (jeśli limit nie został wyczerpany, a test jest nadal aktywny). Uczeń widzi wyłącznie testy aktywne oraz te nieaktywne, które już rozwiązał (zachowując dostęp do swojego wyniku); po dezaktywacji testu niewykorzystane podejścia nie są już dostępne.



Rysunek 8.8. Lista testów przypisanych uczniowi.

8.8. Rozwiązywanie testu

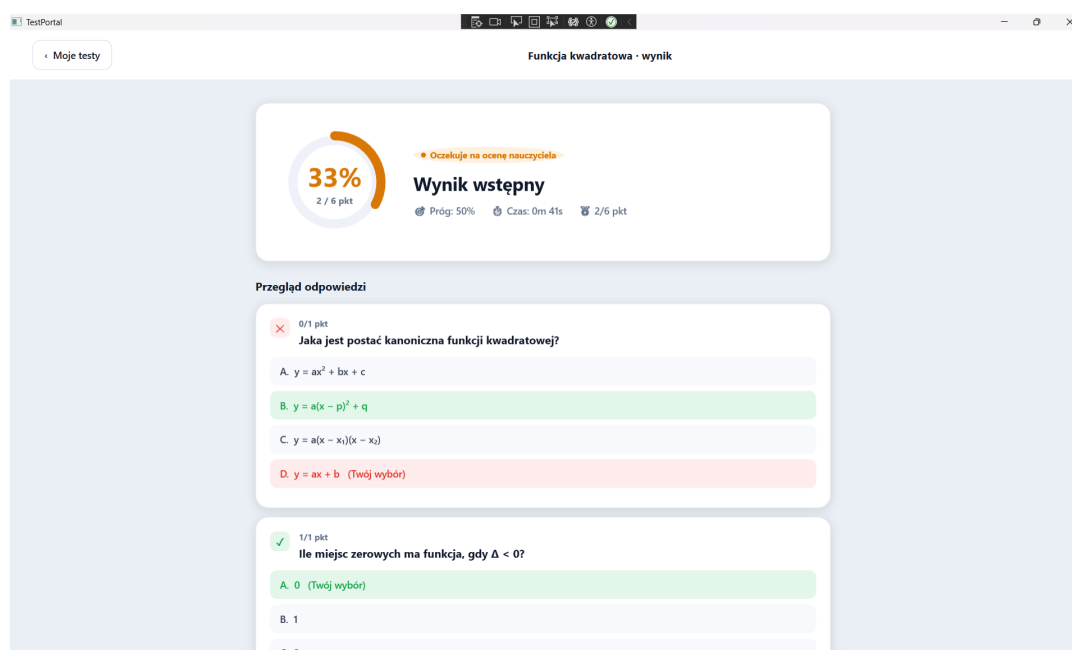
Ekran rozwiązywania testu (rysunek 8.9) prezentuje pojedyncze pytanie wraz z wariantami odpowiedzi lub polem na odpowiedź otwartą. W górnej części znajduje się licznik czasu, a u dołu - nawigacja umożliwiająca przechodzenie między pytaniami oraz pasek postępu. Po upływie czasu test jest automatycznie kończony. Przy oddawaniu testu z pominiętymi pytaniami wyświetlane jest ostrzeżenie. Opuszczenie okna testu jest traktowane jako próba oszustwa.



Rysunek 8.9. Ekran rozwiązywania testu.

8.9. Wynik testu

Po zakończeniu testu uczeń widzi ekran wyniku (rysunek 8.10), prezentujący osiągnięty procent, liczbę zdobytych punktów oraz status (zaliczony, niezaliczony lub oczekujący na ocenę pytań otwartych). Poniżej znajduje się szczegółowy przegląd odpowiedzi - dla każdego pytania wskazane są odpowiedzi poprawne oraz wybrane przez ucznia.

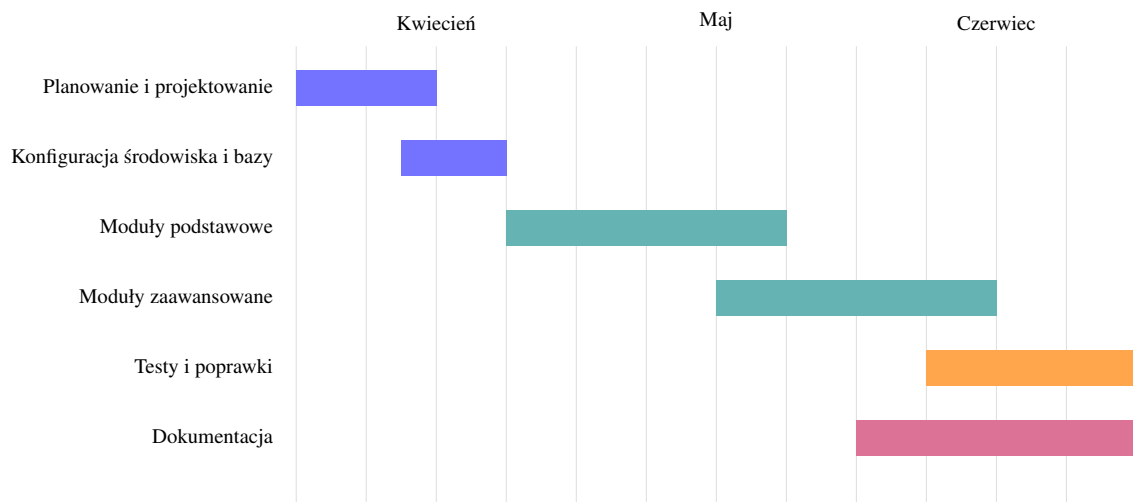


Rysunek 8.10. Ekran wyniku testu wraz z przeglądem odpowiedzi.

9. Harmonogram realizacji projektu

W tym rozdziale przedstawiono harmonogram realizacji projektu, obrazujący poszczególne etapy pracy oraz ich ramy czasowe. Do jego wizualizacji wykorzystano diagram Gantta (rysunek 9.1), który pozwala w czytelny sposób przedstawić planowanie i nakładanie się poszczególnych faz prac.

- **Planowanie i projektowanie** -faza początkowa obejmująca analizę wymagań oraz zaprojektowanie ogólnej struktury aplikacji i schematu bazy danych.
- **Konfiguracja środowiska i bazy danych** -przygotowanie narzędzi, utworzenie projektu oraz konfiguracja połączenia z bazą danych i pierwszych migracji.
- **Implementacja modułów podstawowych** -rozwój podstawowych funkcji: logowania i rejestracji, zarządzania grupami oraz tworzenia testów.
- **Implementacja modułów zaawansowanych** -ocenianie odpowiedzi, obsługa wielu podejść, wykrywanie oszustw oraz statystyki i eksport wyników.
- **Testy i poprawki** -weryfikacja poprawności działania aplikacji oraz eliminacja napotkanych błędów.
- **Tworzenie dokumentacji** -przygotowanie dokumentacji, prowadzone częściowo równoległe z pracami implementacyjnymi.



Rysunek 9.1. Diagram Gantta przedstawiający harmonogram realizacji projektu.

9.1. Napotkane trudności w trakcie realizacji projektu

Podczas realizacji projektu napotkano kilka trudności, które wymagały dodatkowego czasu i uwagi:

- **Mapowanie dziedziczenia na bazę danych** -odwzorowanie trzech hierarchii klas (użytkownicy, pytania, odpowiedzi) w bazie wymagało prawidłowej konfiguracji strategii Table-per-Hierarchy oraz dyskryminatorów w Entity Framework Core.

- **Reguły usuwania powiązanych danych** -konieczne było staranne dobranie reguł kaskadowego i ograniczającego usuwania, tak aby z jednej strony usunięcie testu czyściło powiązane dane, a z drugiej - edycja testu z istniejącymi rozwiązaniami nie naruszała oddanych już odpowiedzi uczniów.
- **Obsługa wielu podejść i wyboru najlepszego wyniku** -wymagała rozróżnienia między zapisem nowego podejścia a aktualizacją oceny istniejącego oraz spójnego liczenia wyników po stronie ucznia i nauczyciela.
- **Wykrywanie prób oszustwa** -konieczne było prawidłowe rozpoznawanie utraty fokusu okna oraz odróżnienie własnych okien dialogowych aplikacji od faktycznego przełączenia się użytkownika na inny program.

9.2. Repozytorium i system kontroli wersji

Kod źródłowy projektu jest przechowywany i zarządzany za pomocą systemu kontroli wersji Git. Dostęp do niego możliwy jest poprzez publiczne repozytorium w serwisie GitHub. Korzystanie z systemu kontroli wersji pozwoliło na śledzenie wszystkich zmian w kodzie oraz zarządzanie kolejnymi wersjami projektu.

Adres repozytorium projektu:

<https://github.com/MichalNocek/ProjektPO2>

10. Instrukcja uruchomienia projektu

10.1. Wymagania systemowe

- **System operacyjny:** Windows 10 lub Windows 11.
- **Środowisko uruchomieniowe:** platforma .NET (wersja zgodna z projektem) ze wsparciem WPF.
- **Serwer bazy danych:** PostgreSQL (zalecana wersja 13 lub nowsza).
- **Dodatkowe narzędzia:** środowisko programistyczne wspierające .NET (np. Visual Studio) oraz narzędzie do zarządzania bazą danych (np. pgAdmin).

10.2. Konfiguracja bazy danych

Aplikacja łączy się z bazą danych PostgreSQL przy użyciu parametrów zdefiniowanych w klasie `AppDbContext`: serwer `localhost`, port `5432`, baza testownik, użytkownik `postgres`. Przed pierwszym uruchomieniem należy zapewnić działający serwer PostgreSQL z dostępem do tych parametrów (w razie potrzeby parametry połączenia można dostosować w kodzie klasy `AppDbContext`).

Ręczne tworzenie bazy ani tabel nie jest wymagane - schemat zakładany jest automatycznie. Przy starcie aplikacja wywołuje metodę `Database.Migrate()`, która tworzy bazę i nakłada wszystkie migracje Entity Framework Core, a następnie klasa `DbSeeder` wypełnia ją danymi początkowymi (przykładowy nauczyciel, uczniowie, grupy oraz testy).

10.3. Konfiguracja środowiska i uruchomienie aplikacji

Kolejne kroki uruchomienia:

1. uruchomić serwer PostgreSQL i upewnić się, że jest dostępny na `localhost:5432`;
2. otworzyć projekt w środowisku Visual Studio (plik rozwiązania `ProjektPO2.slnx`) i uruchomić aplikację (lub wykonać polecenie `dotnet run` w katalogu projektu);
3. przy pierwszym uruchomieniu baza zostanie utworzona i zasilona danymi demonstracyjnymi;
4. zalogować się przy użyciu jednego z kont demonstracyjnych.

10.4. Dane logowania testowego

Po zasileniu bazy danymi początkowymi dostępne są konta demonstracyjne (hasło dla wszystkich: `demo1234`):

- nauczyciel - login `m.dabrowski`;
- uczniowie - loginy `anna.k`, `piotr.n`, `maria.w` oraz kolejni.

11. Podsumowanie

11.1. Ocena stopnia realizacji założeń

Cel projektu został zrealizowany. Powstała działająca aplikacja desktopowa wspierająca nauczyciela i ucznia w pracy z testami, napisana w C# na platformie .NET, z interfejsem WPF i bazą PostgreSQL obsługiwana przez Entity Framework Core. Zrealizowano wszystkie założone funkcje:

- **Konta i role** - rejestracja z wyborem roli i logowanie z rozróżnieniem nauczyciela i ucznia; każdy nauczyciel zarządza wyłącznie własnymi testami.
- **Grupy** - tworzenie grup i zarządzanie ich składem, z przynależnością ucznia do wielu grup.
- **Testy** - tworzenie, edycja, duplikowanie i usuwanie testów z trzema typami pytań, walidacją kompletności oraz sterowaniem widocznością.
- **Rozwiązywanie** - rozwiązywanie w ograniczonym czasie, z obsługą wielu podejść i wyborem najlepszego wyniku.
- **Ocenianie** - automatyczne ocenianie pytań zamkniętych (z punktami częściowymi) oraz ręczne ocenianie i korekta oceny dowolnego podejścia.
- **Bezpieczeństwo i integralność** - haszowanie haseł, unikalność loginów, wykrywanie prób oszustwa, spójność danych zapewniona kluczami obcymi i migracjami.
- **Analiza wyników** - statystyki testu z prezentacją wszystkich podejść oraz eksport do CSV.

W trakcie tworzenia projektu zastosowano zasady programowania obiektowego: dziedziczenie (odzworowane w bazie strategią Table-per-Hierarchy), enkapsulację, polimorfizm oraz podział kodu na niezależne warstwy, dzięki czemu struktura aplikacji jest czytelna i łatwa w utrzymaniu.

11.2. Napotkane problemy

Podczas realizacji projektu napotkano kilka trudności wymagających dodatkowej uwagi:

- **Mapowanie dziedziczenia na bazę danych** - odwzorowanie trzech hierarchii klas wymagało prawidłowej konfiguracji strategii Table-per-Hierarchy oraz dyskryminatorów.
- **Reguły usuwania powiązanych danych** - konieczne było staranne dobranie reguł kaskadowego i ograniczającego usuwania, tak aby usunięcie testu czyściło powiązane dane, a edycja testu z istniejącymi rozwiązaniami nie naruszała oddanych odpowiedzi uczniów.
- **Obsługa wielu podejść** - wymagała rozróżnienia między zapisem nowego podejścia a aktualizacją oceny istniejącego oraz spójnego liczenia najlepszego wyniku.
- **Wykrywanie prób oszustwa** - konieczne było odróżnienie własnych okien dialogowych aplikacji od faktycznego przełączenia się użytkownika na inny program.

11.3. Możliwe kierunki dalszego rozwoju

Choć aplikacja jest w pełni funkcjonalna, można ją w przyszłości rozbudować, na przykład o:

- losowanie kolejności pytań i odpowiedzi w celu dodatkowego utrudnienia ściągania,
- podgląd historii własnych podejść po stronie ucznia,
- terminy dostępności testów (przedział dat, w którym test można rozwiązać),
- rozbudowane raporty i wykresy postępów uczniów w czasie.

Funkcje te nie były jednak konieczne do realizacji założeń projektu, które zostały spełnione w całości.

Bibliografia

- [1] Microsoft. Dokumentacja języka c#. *<https://learn.microsoft.com/dotnet/csharp/>*, 2026. dostęp: 2026.
- [2] Microsoft. Entity framework core – dokumentacja. *<https://learn.microsoft.com/ef/core/>*, 2026. dostęp: 2026.
- [3] Microsoft. Windows presentation foundation (wpf) – dokumentacja. *<https://learn.microsoft.com/dotnet/desktop/wpf/>*, 2026. dostęp: 2026.
- [4] Npgsql Development Team. Npgsql – .net data provider for postgresql. *<https://www.npgsql.org/>*, 2026. dostęp: 2026.
- [5] The PostgreSQL Global Development Group. Postgresql documentation. *<https://www.postgresql.org/docs/>*, 2026. dostęp: 2026.

Spis rysunków

6.1	Diagram ERD bazy danych aplikacji.	16
7.1	Diagram UML klas modelu aplikacji.	19
8.1	Ekran logowania i rejestracji.	22
8.2	Pulpit nauczyciela.	23
8.3	Lista testów nauczyciela ze statusem widoczności i akcjami.	24
8.4	Edytor testu wraz z ustawieniami i listą pytań.	24
8.5	Ocenianie pytań otwartych.	25
8.6	Widok zarządzania grupami uczniów.	26
8.7	Statystyki wyników testu.	27
8.8	Lista testów przypisanych uczniowi.	27
8.9	Ekran rozwiązywania testu.	28
8.10	Ekran wyniku testu wraz z przeglądem odpowiedzi.	29
9.1	Diagram Gantta przedstawiający harmonogram realizacji projektu.	30

Spis tabel

6.1	Najważniejsze tabele bazy danych aplikacji.	15
-----	---	----

Spis listingów

4.1	Struktura katalogów projektu.	12
7.1	Haszowanie hasła i logowanie.	20
7.2	Bezpieczne usuwanie testu (zachowanie integralności).	20
7.3	Ocenianie pojedynczego pytania (GradingService).	21